

توضیح شفاهی پروژه – ۵ دقیقه بک‌اند + ۵ دقیقه فرانت‌اند

متن آماده برای جلسه‌ی دفاع. زبان ساده، بدون اصطلاح اضافه. هر بخش حدود ۵ دقیقه است (~ ۷۰۰ کلمه، سرعت معمولی صحبت).

این فایل برای خود شماست، نه برای نمایش روی پرده. اسلایدها انگلیسی‌اند؛ این متن فارسی است چون قرار است شما آن را بگویید.

بخش اول – بک‌اند (۵ دقیقه)

[0:00] یک جمله‌ی کلیدی برای شروع

«نکته‌ی اصلی معماری KooGYMaa این است که بک‌اند جدا از فرانت‌اند نیست. یک اپلیکیشن Next.js داریم که هم صفحه‌ها را رندر می‌کند، هم API را سرو می‌کند، هم منطق دامنه در آن است. یک پروژه، یک دیپلوی.»

چرا این تصمیم گرفته شد: پروژه یک نفره است. اگر بک‌اند را جدا می‌کردم، باید دو سرویس، دو دیپلوی، احراز هویت بین سرویسی و تعریف تایپ تکراری را نگه می‌داشتم. با App Router، تایپ‌ها بین سرور و کلاینت مشترک‌اند.

[0:40] لایه‌بندی – مهم‌ترین قسمت این بخش

سه لایه داریم و قانون این است که هر لایه فقط با لایه‌ی زیرش حرف می‌زند:

۱. **Route Handler** (`app/api/.../route.ts`) – ۷۶ مورد. فقط سه کار می‌کند: احراز هویت، اعتبارسنجی ورودی، برگرداندن پاسخ. هیچ منطق کسب‌وکاری داخلش نیست.
۲. **لایه‌ی دامنه** (`lib/`) – حدود ۳۰ ماژول. `finance`, `scheduling`, `subscriptions-ops`, `payments/`. منطق واقعی اینجاست و چون از HTTP مستقل است، مستقیم تست می‌شود.
۳. **Prisma** – دیتابیس.

مثال عینی – یک روت کامل (ثبت اندازه‌گیری بدن):

```
const authorization = await authorizeApiRequest(request, ["USER"]);
if (!authorization.ok) return authorization.response;

const validation = validateMeasurement(await request.json());
if (!validation.ok)
  return NextResponse.json({ error: validation.error }, { status: 400 });

await prisma.bodyMeasurement.update({ ... });
```

سه خط. اجازه، اعتبارسنجی، عملیات. هر ۷۶ روت همین شکل‌اند.

[1:40] احراز هویت و کنترل دسترسی

- رمزها با **bcrypt** با ۱۲ cost factor هش می‌شوند.
- بعد از ورود، یک **JWT** با الگوریتم HS256 امضا می‌شود و داخل کوکی **HttpOnly** می‌رود. یعنی جاوااسکریپت مرورگر اصلاً به توکن دسترسی ندارد - این جلوی سرقت نشست با XSS را می‌گیرد.
- کوکی `sameSite: lax` و در پروداکشن `secure` است.
- یک نکته‌ی مهم که معمولاً سؤال می‌شود - «**JWT که قابل ابطال نیست، اگر کاربری را مسدود کنید چه؟**» جوابش در `lib/session.ts` است: هر توکن یک `jwt` دارد و در هر درخواست، جدول `UserSession` چک می‌شود. اگر نشست ابطال شده باشد، توکن رد می‌شود. ضمناً وضعیت کاربر هم از دیتابیس خوانده می‌شود؛ اگر `status` دیگر `ACTIVE` نباشد، دسترسی قطع است.
- نکته‌ی کلیدی: مجوزها هرگز به کلاینت اعتماد نمی‌کنند. نقش داخل توکن هست، ولی در سرور دوباره از دیتابیس خوانده می‌شود. مخفی‌کردن یک دکمه در UI امنیت نیست؛ محافظت واقعی در `authorizeApiRequest` است.

[2:50] پرداخت - جالب‌ترین قسمت بک‌اند

- پرداخت را پشت یک اینترفیس گذاشتم (`PaymentProvider`)، نه مستقیم به درگاه وصل. الان یک `DemoPaymentProvider` دارد؛ برای درگاه واقعی فقط یک کلاس جدید اضافه می‌شود و بقیه‌ی کد دست نمی‌خورد.
- سه مسئله‌ی واقعی که حل شد:
- **دوبار پرداخت شدن (idempotency):** هر `checkout` یک `idempotencyKey` یکتا دارد. اگر کاربر دوبار دکمه را بزند یا شبکه قطع و وصل شود، همان پرداخت قبلی برمی‌گردد، نه یک تراکنش جدید.
 - **اعتماد به وب‌هوک:** پاسخ درگاه با **HMAC-SHA256** امضا و با `timingSafeEqual` مقایسه می‌شود - نه با `===`، تا در برابر حمله‌ی زمان‌سنجی مقاوم باشد.
 - **یکپارچگی داده:** ساخت اشتراک، پرداخت و رکورد ممیزی داخل یک تراکنش دیتابیس انجام می‌شود. یا هر سه ثبت می‌شوند یا هیچ‌کدام.
 - پول هم به صورت **عدد صحیح** ذخیره می‌شود، نه اعشاری - خطای ممیز شناور در محاسبات مالی قابل قبول نیست.

[4:10] دیتابیس و کیفیت

- **Prisma ORM** روی **SQLite/libSQL**. برای توسعه فایل محلی، برای استقرار **Turso** (همان **SQLite** ولی ابری). کد یکسان است.
- **۳۳ مدل و ۱۶ enum**. وضعیت‌ها `enum` نه رشته‌ی آزاد، پس حالت نامعتبر اصلاً قابل ذخیره نیست.
- **مهاجرت‌ها (migrations)** منبع حقیقت‌اند - طرح دیتابیس دستی تغییر نمی‌کند.
- **۷۳ تست خودکار** با **Vitest**، روی همان لایه‌ی دامنه. به‌علاوه `tsc --noEmit` و **ESLint** در **GitHub Actions** روی هر `push`.
- **AuditLog**: هر عمل حساس مدیر و هر تراکنش مالی ثبت می‌شود - چه کسی، چه زمانی، چه چیزی.

[4:50] جمله‌ی پایانی بک‌اند

«خلاصه: منطق در لایه‌ای مستقل از HTTP است تا تست‌پذیر باشد، مجوزها سمت سرور و روی هر درخواست بررسی می‌شوند، و پول با تراکنش و `idempotency` مدیریت می‌شود.»

بخش دوم – فرانت‌اند (۵ دقیقه)

[0:00] شروع

«رابط کاربری ۴۷ صفحه و ۱۰۰ کامپوننت دارد. مهم‌ترین تصمیم فرانت‌اند این بود که پیش‌فرض، کامپوننت سروری باشد و فقط جایی که واقعا تعامل لازم است به کلاینت برود.»

[0:30] Server Components در مقابل Client Components

از ۱۰۰ کامپوننت، ۴۶ تا `"use client"` دارند. یعنی بیش از نیمی از UI هرگز به مرورگر ارسال نمی‌شود.

این یعنی چه؟ صفحه‌ی داشبورد عضو، داده را مستقیماً از دیتابیس می‌خواند و HTML آماده می‌فرستد:

- نه حالت loading، نه اسپینر، نه `useEffect` برای گرفتن داده
 - جاوااسکریپت کمتر – بارگذاری سریع‌تر روی موبایل و اینترنت ضعیف
 - کلید API و کوئری دیتابیس هرگز به کلاینت نشت نمی‌کند
- `"use client"` فقط جایی است که تعامل لازم است: فرم‌ها، نمودار پیشرفت، تایمر استراحت، تقویم جلسات، سوییچ تم.

قانون عملی‌ای که برای خودم گذاشتم: `"use client"` را تا پایین‌ترین سطح ممکن هل بده. مثلاً در صفحه‌ی مدیریت پرداخت‌ها، خود جدول سروری است و فقط دکمه‌های عملیات هر ردیف کلاینت‌اند – نه اینکه کل صفحه کلاینت شود.

[1:30] استایل – چرا کتابخانه‌ی UI استفاده نشد

Tailwind CSS 4 به‌همراه یک فایل `globals.css` با متغیرهای CSS (`--ink`, `--paper`, `--lime`, ...). نه Bootstrap، نه Material UI.

دلیلش: با متغیر CSS، تم تاریک فقط **تغییر مقدار همان متغیرها** است، نه نوشتن دوباره‌ی استایل‌ها. یک کلاس `theme-dark` روی `<html>` و کل برنامه عوض می‌شود.

نمودار پیشرفت هم **SVG دست‌ساز** است، بدون هیچ کتابخانه‌ای – مسیر نمودار با محاسبه‌ی مختصات ساخته می‌شود. یعنی صفر کیلوبایت وابستگی اضافه. کاربر می‌تواند شاخص را عوض کند (وزن، درصد چربی، دور کمر، دور سینه) و بازه را فیلتر کند (۷، ۳۰، ۹۰ روز یا کل)؛ فیلترکردن با `useMemo` انجام می‌شود تا با هر رندر دوباره محاسبه نشود.

در کل پروژه فقط **۹ وابستگی زمان اجرا** وجود دارد. نه کتابخانه‌ی نمودار، نه کتابخانه‌ی فرم، نه کتابخانه‌ی UI. این یعنی باندل کوچک و نبود ریسک منسوخ‌شدن وابستگی‌ها – که برای پروژه‌ای که باید سال‌ها نگهداری شود مهم است.

[2:20] دوزبانه بودن و RTL – سخت‌ترین قسمت

این جدی‌ترین چالش فرانت‌اند بود.

- یک دیکشنری متمرکز در `lib/i18n/translations.ts` با ۵۴۲ کلید.
- زبان در **کوکی** ذخیره می‌شود و در `layout.tsx` **سمت سرور** خوانده می‌شود. برای همین `<html lang dir>` از

همان اولین HTML درست است و صفحه هنگام لود **نمی‌پرد**. اگر این کار سمت کلاینت انجام می‌شد، کاربر یک لحظه چیدمان اشتباه می‌دید.

- `dir="rtl"` یعنی کل چیدمان آینه می‌شود – نه فقط متن. منوها، فلش‌ها، فاصله‌ها.
- دو فونت: **Inter** برای لاتین و **Estedad** برای فارسی. **Estedad** **خودمیزبان** است (فایل woff2 در پروژه)، نه از Google Fonts – هم سریع‌تر، هم بدون وابستگی به سرویس خارجی، هم قابل استفاده در شبکه‌ی محدود.
- نکته‌ی ظریفی که وقت گرفت: در SVG **اعداد باید لاتین باشند**. اگر ارقام فارسی داخل مسیر SVG برود، مرورگر نمی‌تواند آن را پارس کند و نمودار اصلاً رسم نمی‌شود. در کد کامنت گذاشته‌ام.

[3:30] فرم‌ها و تجربه‌ی کاربری

- اعتبارسنجی **دو لایه** است: سمت کلاینت برای بازخورد فوری، سمت سرور برای امنیت. قانون این است که اعتبارسنجی کلاینت فقط راحتی کاربر است؛ تصمیم نهایی همیشه با سرور است.
- خطاها پیام فارسی قابل فهم دارند (`lib/i18n/fa-errors.ts`)، نه کد خطا.
- **دسترس‌پذیری**: از `aria-selected` و `aria-label` استفاده شده و همه‌چیز با کیبورد قابل استفاده است.
- **واکنش‌گرا** تا عرض موبایل؛ منوی کناری در موبایل به منوی بازشو تبدیل می‌شود.
- حالت‌های `loading.tsx`، `error.tsx` و `not-found.tsx` در App Router تعریف شده‌اند، پس کاربر هیچ‌وقت صفحه‌ی سفید یا خطای خام نمی‌بیند.

[4:00] سه فضای کاری، یک پایه‌ی مشترک

عضو، مربی و مدیر هرکدام مسیر و منوی خودش را دارند (`app/admin`، `app/trainer`، `app/user`) ولی روی همان کامپوننت‌های پایه ساخته شده‌اند – جدول داده، فرم، کارت. برای همین ظاهر یکدست است و اضافه‌کردن یک صفحه‌ی جدید به هر نقش ارزان تمام می‌شود.

محافظت مسیرها هم سمت سرور است: `requireCurrentUser(["TRAINER"])` در بالای صفحه، که اگر نقش نخورد کاربر را ریدایرکت می‌کند – قبل از اینکه حتی یک بایت HTML تولید شود.

[4:20] امنیت سمت مرورگر

در `next.config.ts` هدرهای امنیتی تنظیم شده‌اند: **CSP**، `X-Frame-Options: DENY` (ضد clickjacking)، `Referrer-Policy` و `nosniff` و در پروداکشن **HSTS**.

[4:45] جمله‌ی پایانی فرانت‌اند

«خلاصه: پیش‌فرض سروری برای سرعت، کلاینت فقط جایی که تعامل هست، تم و زبان سمت سرور تا صفحه نپرد، و بدون وابستگی سنگین UI.»

سؤال‌های محتمل داور

چرا SQLite و نه PostgreSQL؟ حجم داده‌ی این دامنه کوچک است و libSQL/Turso همان SQLite را با replication ابری می‌دهد. چون از Prisma استفاده شده، مهاجرت به Postgres تغییر provider و migration است، نه بازنویسی کد.

چرا Redux/Zustand نیست؟ چون بیشتر داده‌ها روی سرور رندر می‌شوند، حالت سراسری کلاینت تقریباً لازم نشد. فقط تم و زبان Context دارند.

مقیاس‌پذیری؟ لایه‌ی دامنه از HTTP مستقل است، پس جداکردن آن به سرویس مجزا ممکن است. گلوگاه فعلی دیتابیس است، نه کد. محدودکننده‌ی نرخ (rate limit) الان درون حافظه‌ای است و برای چند نمونه‌ای باید به Redis منتقل شود - این را به‌عنوان کار آینده می‌شناسم.

بزرگ‌ترین چالش؟ همزمانی در پرداخت. حل با idempotency key و تراکنش دیتابیس.